Comparing First-Order Methods for L1-Regularized Least Squares

1st Meixuan Chen
Edward S. Rogers Sr. Department of Electrical & Computer Engineering
University of Toronto
Toronto, Canada
meixan.chen@mail.utoronto.ca

***Abstract*—This paper empirically compare three first-order methods for L1-regularized least squares (LASSO): ISTA, FISTA, and ADMM. Using synthetic datasets with varying conditioning and a small real-world regression task, we evaluate convergence speed (iterations and wall-clock time), hyperparameter sensitivity, and implementation trade-offs. FISTA converges fastest overall and remains robust to step-size choices near the Lipschitz-based value. ADMM is more iteration-efficient on ill-conditioned problems but highly sensitive to its penalty parameter, while ISTA provides a simple baseline. These results offer practical guidance for choosing first-order solvers in high-dimensional sparse regression.**

***Index Terms*—LASSO, sparse regression, convex optimization, ISTA, FISTA, ADMM, first-order methods, convergence analysis, hyperparameter sensitivity**

I. Introduction

L1-regularized least squares (LASSO) is a standard convex formulation for sparse linear regression. It was introduced in the foundational work of Tibshirani et al.[1] as a method for simultaneous shrinkage and subset selection in linear models.

Because LASSO jointly supports prediction and feature selection, it has broad real-world impact across fields where high-dimensional data and interpretability are both important. In clinical medicine, LASSO has been used to identify key risk factors for cardiovascular comorbidities in COPD patients[2]. In finance, it underpins predictive models for stock returns and macroeconomic indicators, where selecting relevant predictors from large datasets improves forecasting accuracy while preserving interpretability[3]. More recently, LASSO-based methods have been used in public health to forecast infectious disease outbreaks, such as Mpox, by identifying cross-jurisdictional predictors that strengthen early warning systems[4].

These examples illustrate LASSO's versatility: it not only delivers competitive predictive accuracy but also yields sparse models that highlight the most influential variables, a feature that is particularly valuable when interpretability and actionable insight are as important as prediction itself. Therefore, investigating the computational tools needed to implement LASSO efficiently in these real-world settings is essential.

Standard methods for LASSO include proximal gradient descent (ISTA), its accelerated variant FISTA, and operator-splitting approaches such as ADMM. These methods have well-understood convergence guarantees (e.g., $O(\frac{1}{k})$ for ISTA and $O(\frac{1}{k^2})$ for FISTA in terms of objective gap) and are widely used in signal processing and machine learning. Standard references (e.g., [5]; [6]; [7]) describe the algorithms and their high-level theory, but they typically do not provide a systematic, side-by-side empirical comparison across varying conditioning and parameter choices. This leaves a gap between clean theory and the behavior observed in practical implementations.

This paper provides an empirical study of several classical algorithms for LASSO and compares their convergence behavior and practical performance on synthetic and small real datasets, with particular attention to how conditioning and parameter choices affect results. Specifically, we evaluate the following research questions:

- **RQ1**: How do ISTA, FISTA, and ADMM compare in convergence speed (iterations and wall-clock time) across LASSO problems with different conditioning and on a small real dataset?
- **RQ2**: How sensitive are these methods to hyperparameters such as the step size (for ISTA/FISTA) and the penalty parameter ρ (for ADMM)?
- **RQ3**: In practice, what are the trade-offs among implementation simplicity, tuning effort, and robustness for these first-order methods?

Our results show that ISTA is the simplest to implement and serves as a useful baseline, but it is the slowest in both iterations and wall-clock time and is the most sensitive to problem conditioning. FISTA, which adds a momentum term, achieves the fastest convergence in both iterations and time while requiring essentially the same step-size tuning as ISTA, making it a robust default for composite convex optimization. ADMM, though more complex because of variable splitting and linear-system solves, offers competitive iteration counts; its performance depends critically on the penalty parameter ρ, and it can be less sensitive to conditioning, making it particularly suitable for constrained, structured, or distributed problems.

II. Algorithms and Problem Setup

A. Problem Setup

The LASSO problem is defined as follows: given a data matrix $A \in \mathbb{R}^{n\times d}$ and response vector $b \in \mathbb{R}^n$, the goal is to

find a coefficient vector x that both fits the data and promotes sparsity:

$$\min_{x\in\mathbb{R}^d} \frac{1}{2}\|Ax-b\|_2^2 + \lambda\|x\|_1. \tag{1}$$

This problem is convex but non-smooth, and is typically solved by first-order methods that exploit the composite structure f(x)=g(x)+h(x) with smooth g and non-smooth h.

B. Algorithms

We implemented and evaluated the following algorithms for the LASSO problem.

1) ISTA: The **Iterative Shrinkage-Thresholding Algorithm (ISTA)** is a first-order proximal gradient method for solving the LASSO problem by minimizing the composite objective

$$F(w) = \frac{1}{2}\|Xw-y\|_2^2 + \lambda\|w\|_1. \tag{2}$$

The algorithm alternates between a gradient step on the smooth least-squares term and a proximal (soft-thresholding) step for the nonsmooth ℓ_1 penalty. Given a step size η (the reciprocal of the Lipschitz constant L), the update is

$$w^{(k+1)} = S_{\eta\lambda}\left(w^{(k)} - \eta X^T(Xw^{(k)}-y)\right), \tag{3}$$

where $S_\kappa(a) = \operatorname{sign}(a)\max(|a|-\kappa, 0)$ is the soft-thresholding operator. The per-iteration cost is dominated by matrix-vector products, yielding complexity $O(d^2)$ or $O(nd)$, depending on how the gradient is implemented. ISTA has a sublinear convergence rate of $O(1/k)$, where k is the iteration count.

2) FISTA: The **Fast Iterative Shrinkage-Thresholding Algorithm (FISTA)**[5] improves ISTA by incorporating Nesterov momentum. Instead of applying the proximal update directly to $w^{(k)}$, FISTA updates an auxiliary variable $z^{(k)}$, formed as a linear combination of recent iterates. The momentum parameter is updated by

$$t_{k+1} = \frac{1+\sqrt{1+4t_k^2}}{2}, \tag{4}$$

and the extrapolated point is

$$z^{(k)} = w^{(k)} + \left(\frac{t_k-1}{t_{k+1}}\right)(w^{(k)} - w^{(k-1)}). \tag{5}$$

The proximal step then follows the same form as ISTA, but applied at $z^{(k)}$. While the per-iteration complexity remains $O(d^2)$, the global convergence rate improves to $O(1/k^2)$, making FISTA substantially faster in practice for large-scale problems.

3) ADMM: The **Alternating Direction Method of Multipliers (ADMM)**[8] solves LASSO through variable splitting and an augmented Lagrangian formulation. The problem is rewritten as minimizing $f(w)+g(z)$ subject to $w-z=0$, which leads to three updates per iteration: a w-update (linear solve), a z-update (proximal step), and a u-update (dual variable update). The w-update is

$$w^{(k+1)} = (X^TX+\rho I)^{-1}(X^Ty + \rho(z^{(k)}-u^{(k)})), \tag{6}$$

and the z-update is

$$z^{(k+1)} = S_{\lambda/\rho}(w^{(k+1)} + u^{(k)}). \tag{7}$$

The initial factorization of $X^TX+\rho I$ costs $O(d^3)$, while subsequent iterations can be reduced to $O(d^2)$ via back-substitution. ADMM is especially useful in constrained or distributed settings and often reaches moderate accuracy in a relatively small number of iterations.

C. Implementation Choices

In our implementation, all three algorithms use fixed hyperparameters rather than adaptive line-search strategies. For ISTA and FISTA, we use a constant step size, with default value $1/L$, where $L = \lambda_{\max}(\mathbf{X}^\top\mathbf{X})$ is the largest eigenvalue of $\mathbf{X}^\top\mathbf{X}$. In the step-size sensitivity analysis (RQ3), we vary a scaling factor α and use α/L, with $\alpha \in \{0.1, 0.5, 0.9, 1.0, 1.1\}$. This setup isolates the effect of step size on convergence without additional confounding factors. ADMM uses a fixed penalty parameter ρ: baseline and real-data experiments set $\rho = 1.0$, while RQ3 explores $\rho \in \{0.01, 0.1, 1, 10, 100\}$ to evaluate sensitivity.

We apply a unified stopping criterion across all algorithms. Termination occurs when the relative change in objective value falls below 10^{-6}, i.e.,

$$\frac{|f_k - f_{k-1}|}{f_{k-1} + 10^{-8}} < 10^{-6},$$

or when a maximum of 5000 iterations is reached. This criterion is algorithm-agnostic and does not depend on problem-specific optimality measures, such as gradient-mapping norms or known reference solutions. For ADMM, primal and dual residuals ($\|\mathbf{w}-\mathbf{z}\|$ and $\rho\|\mathbf{z}_k-\mathbf{z}_{k-1}\|$) are computed and logged for diagnostics but are not used as stopping thresholds. This design ensures a consistent and computationally lightweight stopping rule across methods, enabling a fair comparison under a common convergence tolerance.

III. Experimental Setup

The experimental framework is implemented in Python, with custom NumPy implementations of ISTA, FISTA, and ADMM, plus drivers for synthetic and real-world datasets. The stack is lightweight: NumPy/SciPy for computation, Matplotlib for visualization, and scikit-learn for data loading, preprocessing, train-test splitting, and evaluation. The key components, including data generation and metric collection, are outlined below.

A. Synthetic Data Generation

To evaluate the numerical stability and convergence behavior of the LASSO optimization algorithms, we synthesize high-dimensional datasets under controlled spectral and structural conditions. The data-generation process constructs a design matrix $A \in \mathbb{R}^{n\times d}$ and an observation vector $b \in \mathbb{R}^n$ from a sparse ground-truth vector $x^\star$.

a) Design Matrix: We consider two regimes for the design matrix A to test algorithmic robustness under ill-conditioning:

- **Controlled i.i.d. Gaussian Design**
  In this regime, entries of A are sampled from an i.i.d. Gaussian distribution and then normalized column-wise to unit ℓ_2 norm. To control the condition number $\kappa(A)$ precisely, we perform singular value decomposition (SVD) and linearly rescale the singular values between a maximum $\sigma_{\max}$ and a minimum $\sigma_{\min} = \sigma_{\max}/\kappa$. This construction enforces the target condition number while preserving a Gaussian-like structure.
- **Correlated Toeplitz Design**
  To simulate a correlated feature space, we construct A using a Toeplitz covariance structure. The covariance matrix Σ is defined by $\Sigma_{ij} = \rho^{|i-j|}$, where the correlation coefficient ρ is derived from the desired condition number κ via $\rho = (\kappa - 1)/(\kappa + 1)$. Rows of A are then sampled from a multivariate Gaussian distribution $N(0, \Sigma)$, introducing strong linear dependencies among adjacent features.

b) Ground Truth and Observation Model: The ground-truth vector $x^\star$ is generated with sparsity level k, where the support is selected uniformly at random. Nonzero coefficients are drawn from a Rademacher distribution $\{\pm 1\}$ to avoid bias in coefficient magnitudes. The observation vector is defined as

$$b = Ax^\star + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2 I), \tag{8}$$

where ε denotes additive Gaussian noise controlled by the parameter `noise_level` σ.

B. Real-world dataset

We use a small real-world regression dataset from scikit-learn for external validation. The real-data experiment uses the sklearn diabetes dataset, which has $d = 10$ features and $n = 442$ samples overall; the code then applies an 80/20 train-test split, so the optimization runs on the training set of about 353 samples. Feature standardization is done with StandardScaler by calling `fit_transform` on the full feature matrix before splitting, so each feature is centered and scaled to unit variance.

The regularization parameter is chosen from the training data using the standard LASSO reference value

$$\lambda_{\max} = \frac{\max(|X_{\text{train}}^T y_{\text{train}}|)}{n_{\text{train}}}$$

, and then the experiment sets $\lambda = 0.1 * \lambda_{\max}$. All methods use the same data-driven lambda at one-tenth of the maximum value that would shrink the solution most aggressively.

C. Evaluation metrics

The evaluation framework records core metrics through a dedicated `Recorder` callback implemented in `src/experiments.py` and auxiliary functions in `src/metrics.py`. At each iteration, the LASSO objective

$$f(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda\|\mathbf{w}\|_1$$

is stored, enabling direct generation of objective-versus-iteration and objective-versus-time curves from the logged histories. Additionally, the residual norm $\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2$ and sparsity—defined as the number of coefficients whose absolute value exceeds 10^{-6}—are recorded. Wall-clock time is measured using `time.perf_counter()` at the start of each solver run and accumulated per iteration, yielding a time-to-convergence metric. Convergence is determined by a unified early-stopping rule: the process halts when the relative change in the objective falls below 10^{-6} (i.e., $|f_k - f_{k-1}|/(f_{k-1} + 10^{-8}) < 10^{-6}$) or when a maximum of 5000 iterations is reached. For each algorithm and parameter configuration, the time at which this stopping condition is met is collected over multiple trials; aggregated statistics (mean and standard deviation) are reported as summary fields.

IV. Results

In order to systematically compare the performance of each algorithm on the LASSO problem, we evaluated the following research questions:

- **RQ1**: How do ISTA, FISTA, and ADMM compare in convergence speed (iterations and wall-clock time) across LASSO problems with different conditioning and on a small real dataset?
- **RQ2**: How sensitive are these methods to hyperparameters such as the step size (for ISTA/FISTA) and the penalty parameter ρ (for ADMM)?
- **RQ3**: In practice, what are the trade-offs among implementation simplicity, tuning effort, and robustness for these first-order methods?

A. RQ1: Convergence and performance across conditioning & real data

In order to answer RQ1, we designed an experiment using the same synthetic dataset, a unified stopping criterion and fixed hyperparameter policy across all solvers: ISTA and FISTA use a constant step size $1/L$, where L is the largest eigenvalue of $\mathbf{X}^\top\mathbf{X}$; ADMM uses a fixed penalty parameter $\rho = 1$. Termination condition as described in section III.

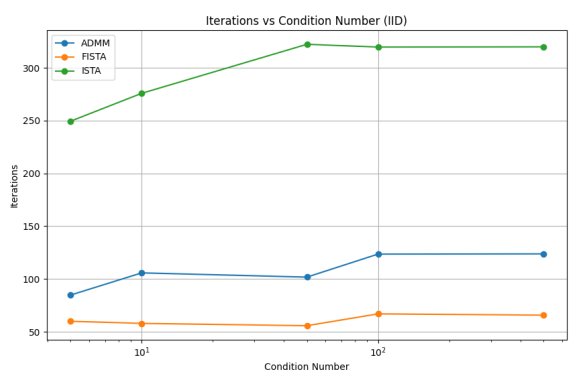

Fig. 1. Iteration vs. Condition Number i.i.d.

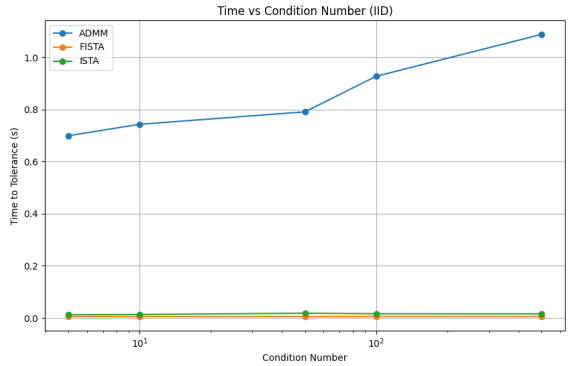

Fig. 2. Time vs. Condition Number i.i.d.

As shown in Fig. 1 and Fig. 2, on synthetic problems with an i.i.d. design matrix ($n = 100$, $d = 200$), FISTA consistently achieves the fastest convergence: across condition numbers ranging from 5 to 500, it requires approximately 56–67 iterations and 0.005–0.006 s of wall-clock time. In contrast, ISTA requires 249–322 iterations and 0.012–0.018 s, while ADMM converges in 85–124 iterations but incurs a significantly higher per-iteration cost, resulting in wall-clock times of 0.70–1.09 s. Hence, for the i.i.d. case, FISTA is superior in both iteration count and elapsed time; ADMM outperforms ISTA in iterations but is substantially slower in runtime, and ISTA is the slowest among the three.

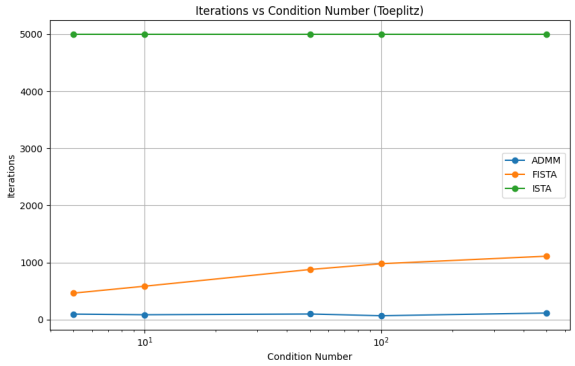

Fig. 3. Condition Number vs. Iterations toeplitz

The performance landscape changes when the design matrix exhibits a correlated Toeplitz structure. Under such ill-conditioned conditions, ADMM demonstrates the greatest robustness in iteration count, converging in 69–116 iterations as the condition number increases. FISTA exhibits a substantial slowdown, requiring 465–1111 iterations, and ISTA fails to meet the stopping tolerance within the 5000-iteration limit for all Toeplitz configurations. Nevertheless, when measured by wall-clock time, FISTA remains advantageous: despite its increased iteration count, it finishes in 0.025–0.059 s, compared to ADMM's 0.61–0.92 s and ISTA's 0.22–0.24 s. This indicates that while ADMM is the most iteration-efficient

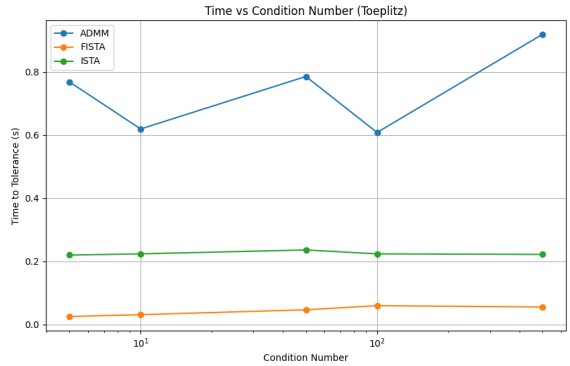

Fig. 4. Condition Number vs. Time toeplitz

under poor conditioning, FISTA still provides the best practical runtime performance in the current NumPy implementation.

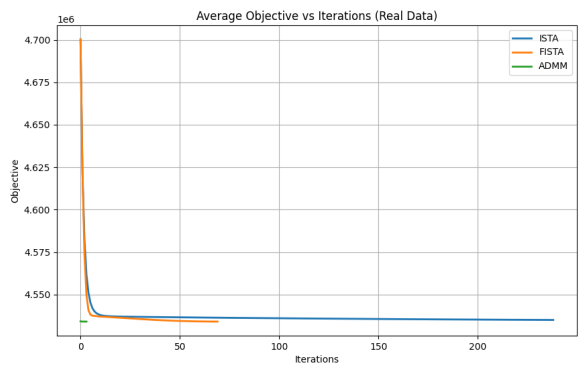

Fig. 5. Real-world dataset Objective Value vs. Iterations

On a small real-world dataset, all three methods converge rapidly, with ADMM achieving the best overall metrics: it averages 3 iterations and 0.00065 s, compared to 69 iterations and 0.0027 s for FISTA, and 238 iterations and 0.0085 s for ISTA. The resulting model quality as measured by test-set prediction metrics is nearly identical across the three solvers, implying that the primary difference lies in optimization speed rather than final solution accuracy.

Taken together, these results support a nuanced recommendation: FISTA emerges as the most effective general-purpose method in terms of wall-clock time on the synthetic benchmarks; ADMM is superior in iteration count and exhibits greater stability on severely correlated problems; and on the small real dataset, ADMM outperforms the others due to the low cost of the linear-system solves at modest dimensionality.

B. RQ2: Hyperparameter sensitivity

The hyperparameter sensitivity of the three algorithms was assessed on a synthetic i.i.d. problem ($n = 100$, $d = 200$, condition number 50, $\lambda = 0.1$). ISTA and FISTA used a step size α/L, where $L = \lambda_{\max}(\mathbf{X}^\top \mathbf{X})$, with $\alpha \in \{0.1, 0.5, 0.9, 1.0, 1.1\}$; ADMM employed a fixed penalty

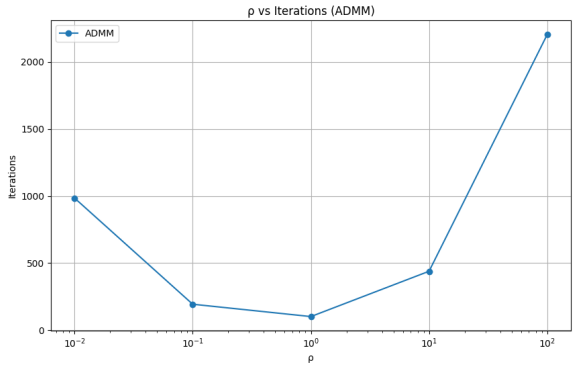

Fig. 6. ADMM rho vs. Iterations

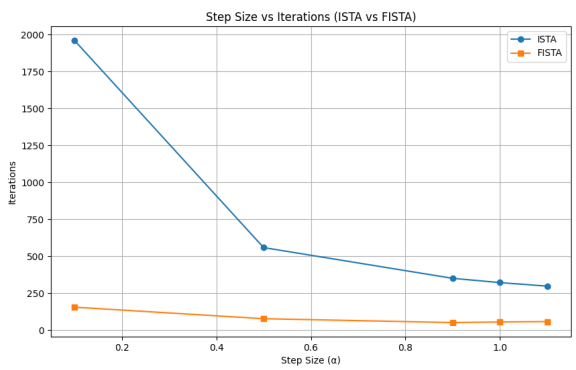

Fig. 7. FISTA and ISTA step size vs. Iterations

$\rho \in \{0.01, 0.1, 1, 10, 100\}$. All runs used the same stopping criterion (relative objective change $< 10^{-6}$).

ISTA was proved to be highly sensitive to the step size. Reducing α from 1.0 to 0.1 increased the mean iteration count from 322 to 1961, and runtime from 0.017 s to 0.088 s. FISTA also benefited from larger steps but remained robust near the standard choice $1/L$: iterations decreased from 156 at $\alpha = 0.1$ to 52 at $\alpha = 0.9$, with runtimes ranging from 0.010 s to 0.005 s. In contrast, ADMM exhibited extreme sensitivity to ρ. The optimal value was $\rho = 1$, yielding 102 iterations and 0.94 s. Deviating from this value caused performance to degrade markedly; for $\rho = 0.01$ and $\rho = 100$, iterations rose to 985 and 2204, respectively, with corresponding runtime increases to 7.38 s and 17.9 s.

These results underscore that while both proximal-gradient methods benefit from steps near $1/L$, FISTA is substantially more robust to suboptimal choices. ADMM's convergence, by contrast, hinges critically on proper tuning of the penalty parameter; a poor choice can degrade performance by more than an order of magnitude.

C. RQ3: Practical trade-offs

A detailed summary is shown in Table I, while ISTA provides the most straightforward implementation, FISTA is generally preferred for large-scale LASSO problems due to its accelerated $O(1/k^2)$ convergence rate. ADMM remains the most robust against the high condition numbers and correlated designs with carefully tuned hyperparameter.

TABLE I
COMPARISON OF OPTIMIZATION ALGORITHMS FOR LASSO REGRESSION

Method	Complexity	Tuning	Convergence	Robustness	Comments
ISTA	Very simple (few lines)	Step size $\alpha \leq 1/L$	Slowest in iterations & time	Most sensitive to conditioning	Excellent baseline and pedagogical tool.
FISTA	Slightly more complex (z_k, t_k)	Same α as ISTA, requires L	Fastest in iterations and time	Improved robustness vs. ISTA	Standard choice for composite convex problems.
ADMM	Most complex (split variables)	Penalty ρ requires tuning	Moderate iterations; heavier cost	High robustness to ill-conditioning	Optimal for constrained or distributed settings.

V. DISCUSSION AND CONCLUSION

A. Summary

Taken together, the findings from RQ1 to RQ3 support a nuanced recommendation. On large-scale synthetic benchmarks, FISTA delivers the best wall-clock performance because its accelerated $O(1/k^2)$ convergence combines with good robustness to step-size choices near the Lipschitz-based value $1/L$. ADMM is more iteration-efficient and often more stable under severe correlation, but it is highly sensitive to the penalty parameter ρ; poor tuning can degrade performance by more than an order of magnitude. On the small real dataset, ADMM achieves the fastest runtime because its per-iteration overhead is negligible at this scale. Overall, ISTA remains the simplest baseline, FISTA is the most reliable default for most LASSO settings, and ADMM is most attractive for ill-conditioned or structured problems when hyperparameters are tuned carefully.

B. Limitations and Future Work

While this study provides a systematic comparison of ISTA, FISTA, and ADMM for the LASSO problem, several limitations should be acknowledged. First, the experimental scope is confined to problems of moderate scale ($n, d \leq 1000$), which, while sufficient for illustrating algorithmic behavior, does not fully capture performance in truly large-scale settings (e.g., $n \gg 10^4$ or $d \gg 10^4$). Second, the investigation is restricted to the standard ℓ_1-regularized least-squares formulation; extensions to other convex penalties (e.g., group LASSO, elastic net) or different loss functions (e.g., logistic regression) were not considered. Third, only one small real-world dataset was included, limiting the generalizability of the findings to diverse application domains.

Future work should extend this study to larger high-dimensional settings (e.g., via randomized linear algebra or distributed implementations), broader models (e.g., group

LASSO and sparse logistic regression), and stronger tuning strategies (e.g., backtracking line search and automatic ADMM penalty selection). Evaluating more public regression and classification datasets would improve external validity, while GPU profiling and just-in-time compilation (e.g., Numba) could clarify hardware-aware performance trade-offs for large-scale sparse learning.

REFERENCES

[1] R. Tibshirani, "Regression shrinkage and selection via the Lasso," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 58, no. 1, pp. 267–288, 1996. [Online]. Available: https://www.jstor.org/stable/2346178

[2] Z. Chen et al., "Development and validation of a nomogram prediction model for cardiovascular comorbidities in COPD patients based on hypertension," *Frontiers in Medicine*, 2025, PMID: 41859166. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/41859166/

[3] Z. Gao et al., "LASSO inference for high dimensional predictive regressions," *arXiv preprint arXiv:2409.10030*, 2024. [Online]. Available: https://arxiv.org/abs/2409.10030

[4] H. Craddock et al., "Early warning of Mpox outbreaks in U.S. jurisdictions using Lasso Vector Autoregression models with cross-jurisdictional lags," *arXiv preprint arXiv:2602.06135*, 2026. [Online]. Available: https://arxiv.org/abs/2602.06135

[5] A. Beck and M. Teboulle, "A fast iterative shrinkage-thresholding algorithm for linear inverse problems," *SIAM Journal on Imaging Sciences*, vol. 2, no. 1, pp. 183–202, 2009. DOI: 10.1137/080716542 eprint: https://doi.org/10.1137/080716542. [Online]. Available: https://doi.org/10.1137/080716542

[6] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Found. Trends Mach. Learn.*, vol. 3, no. 1, pp. 1–122, Jan. 2011, ISSN: 1935-8237. DOI: 10.1561/2200000016 [Online]. Available: https://doi.org/10.1561/2200000016

[7] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, UK: Cambridge University Press, 2004, ISBN: 9780521833783.

[8] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Foundations and Trends in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011.